SIMATS ENGINEERING (SAVEETHASCHOOLOFENGINEERING)

Department of Artificial Intelligence and Data Science

ASSESSMENTTOOL3–CODINGCHALLENGE

Course Code:	DSA03	**Course Title:**	Natural Language Processing
CO Assessed:	CO1 – Imitation (L1)	**Assessment:**	Assessment Tool 3 – Coding Challenge
Weightage:	30% of CIA		
CO5 Statement:	Analyse NLP models, regular expression patterns, finite state automata, morphological processing techniques and to interpret language processing. (BL-3)		
Student Name:	B.V.N.S.BHIMESWAR	**Reg. No.:**	192472048
Section / Batch:	CSE-- AIML	**Date:**	23\7\2026

Code of Conduct

I am B.V.N.S.BHIMESWAR/192472048certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action. Signature of the Student: ____ *B.V.N.S.BHIMESWAR.*_____

Instructions

- Develop a complete and modular Python program that meets all the functional requirements specified in the coding challenge using appropriate algorithms, data structures, and programming practices.
- Test the program using multiple valid and invalid test cases, and clearly display the input, processing, and output to verify the correctness of the implementation.
- Follow standard coding practices by using meaningful variable names, proper indentation, comments, exception handling (where applicable), and upload the source code, sample outputs, and README file to the designated GitHub repository.
- Duration: 40 minutes.

Section / Topic	Focus	Q Nos
Regular Expression Pattern Matching	Regular Expressions, Basic Regular Expression Patterns	1
Finite State Automata Implementation	Finite State Automata, Models and Algorithms	2
Advanced Regular Expression Search	Regular Expressions, Basic Regular Expression Patterns	3

Annexure A – Coding Challenge

Section 1: Intelligent Email and Password Validator using Regular Expressions

1. Problem Statement:

A software company is developing a user registration system. Before creating an account, the system must validate user credentials based on predefined security policies.

Develop a Python application that validates:

Email Address
Password
Mobile Number

using Regular Expressions.

Validation Rules

Email
Must start with an alphabet.
May contain letters, digits, '.', '_' before '@'.
Domain should contain only alphabets.
Valid extensions: .com, .org, .edu, .net, .in

Password

Minimum 8 characters
At least one uppercase letter
At least one lowercase letter
At least one digit
At least one special character (@,#,$,%,&,!)

Mobile Number

Must contain exactly 10 digits.
First digit should be between 6–9.

Expected Output

Display:

Valid Email / Invalid Email
Strong Password / Weak Password
Valid Mobile Number / Invalid Mobile Number

Section 2: Design a Finite State Automata (FSA) Simulator

2. Problem Statement

Develop a Python-based Deterministic Finite Automata (DFA) simulator.

The program should:

- Accept the DFA description ▪ States
  - ▪ Input alphabet
  - ▪ Transition table
  - ▪ Initial state ▪ Final state(s)
- Accept multiple input strings.
- Simulate state transitions.
- Display whether the string is Accepted or Rejected.

Example

Language:
Strings ending with "ab"

Input

abaab

Output

Transition Path:
q0 → q1 → q0 → q1 → q2 Accepted

Section 3: Smart Pattern Matching Engine using Regular Expressions

3. Problem Statement

Develop a Python-based text search engine that performs pattern matching using Regular Expressions.

The system should support:
Word Search
Prefix Search
Suffix Search
Date Search
Phone Number Search
Hashtag Search
Mention (@username) Search

Example Input
Meeting on 12/09/2026
Call 9876543210
#NLP
@OpenAI
natural language processing

User Menu
1.Search Date
2.Search Phone Number
3.Search Hashtag
4.Search Mention
5.Search Prefix
6.Search Suffix

Expected Output
Display all matching patterns based on user selection.

Section 1: User Credential Validation Using Regular Expressions

AIM

To develop a Python application that validates Email Address, Password, and Mobile Number using Regular Expressions.

ALGORITHM

1. **Start the program.**
2. **Import the re module.**
3. **Read Email, Password, and Mobile Number.**
4. **Validate Email using the given pattern.**
5. **Validate Password based on security rules.**
6. **Validate Mobile Number using the given pattern.**
7. **Display validation results.**
8. **Stop the program.**

PYTHON PROGRAM

```
import re

email = "john_123@gmail.com"
password = "Abc@1234" mobile
= "9876543210"

# Email Validation email_pattern = r'^[A-Za-z][A-Za-z0-9._]*@[A-Zaz]+\.(com|org|edu|net|in)$'

if re.fullmatch(email_pattern, email):
    print("Valid Email") else:
    print("Invalid Email")

# Password Validation password_pattern = r'^(?=.*[A-Z])(?=.*[a-
z])(?=.*\d)(?=.*[@#$%&!]).{8,}$'

if re.fullmatch(password_pattern, password):
    print("Strong Password") else:
    print("Weak Password")

# Mobile Validation mobile_pattern
= r'^[6-9]\d{9}$'

if re.fullmatch(mobile_pattern, mobile):
    print("Valid Mobile Number") else:
    print("Invalid Mobile Number")
```

SAMPLE INPUT

Email : john_123@gmail.com
Password : Abc@1234
Mobile : 9876543210

SAMPLE OUTPUT

Valid Email
Strong Password
Valid Mobile Number

RESULT

Thus, the Email Address, Password, and Mobile Number were successfully validated using Regular Expressions.

Section 2: Deterministic Finite Automata (DFA) Simulator AIM

To develop a Python-based DFA Simulator that accepts or rejects input strings based on state transitions.

ALGORITHM

1. Start the program.
2. Define states, alphabet, transitions, initial state, and final state.
3. Read the input string.
4. Begin from the initial state.
5. Process each symbol using the transition table.
6. Display the transition path.
7. Check whether the final state is reached.
8. Display Accepted or Rejected.
9. Stop the program.

PYTHON PROGRAM

```
# DFA for strings ending with "ab"

transitions = {
   ('q0', 'a'): 'q1',
   ('q0', 'b'): 'q0',
   ('q1', 'a'): 'q1',
   ('q1', 'b'): 'q2',
   ('q2', 'a'): 'q1',
   ('q2', 'b'): 'q0'
}

start_state = 'q0' final_state
= 'q2' string =

"abaab"

current = start_state path
= [current]

for ch in string:
   current = transitions[(current, ch)]
path.append(current) print("Transition Path:") print("
→ ".join(path))

if current == final_state:
print("Accepted") else:
   print("Rejected")
```

SAMPLE INPUT::

abaab

SAMPLE OUTPUT

Transition Path:

q0 → q1 → q0 → q1 → q1 → q2

Accepted

RESULT

Thus, the DFA Simulator was successfully implemented to simulate state transitions and determine whether a string is accepted or rejected.

Section 3: Smart Pattern Matching Engine Using Regular Expressions

AIM

To develop a Python-based text search engine using Regular Expressions for pattern matching.

ALGORITHM

1. **Start the program.**
2. **Import the re module.**
3. **Read the text data.**
4. **Display the search menu.**
5. **Perform the selected pattern search.**
6. **Display matching patterns.**
7. **Stop the program.**

PYTHON PROGRAM

```
import re

text = """
Meeting on 12/09/2026
Call 9876543210
#NLP
@OpenAI
natural language processing
"""
print("1.Search Date") print("2.Search
Phone    Number")    print("3.Search

Hashtag") print("4.Search Mention")

print("5.Search                    Prefix")

print("6.Search Suffix") choice = 1

if choice == 1:
   result = re.findall(r'\d{2}/\d{2}/\d{4}', text)
```

```
print("Dates:", result)

elif choice == 2:
    result = re.findall(r'[6-9]\d{9}', text)
print("Phone Numbers:", result)

elif choice == 3:
    result = re.findall(r'#\w+', text)
    print("Hashtags:", result)

elif choice == 4:
    result = re.findall(r'@\w+', text)
print("Mentions:", result)

elif choice == 5:
    result = re.findall(r'\bnat\w*', text)
print("Prefix Matches:", result)

elif choice == 6:
    result = re.findall(r'\w+ing\b', text)
print("Suffix Matches:", result)
```

SAMPLE INPUT

Meeting on 12/09/2026

Call 9876543210

#NLP

@OpenAI

natural language processing

SAMPLE OUTPUT (Choice = 1)

Dates: ['12/09/2026']

RESULT

Thus, the Smart Pattern Matching Engine was successfully implemented using Regular Expressions to perform Date, Phone Number, Hashtag, Mention, Prefix, and Suffix searches

Rubrics for Calculation

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20%	Clearly understands problem constraints, edge cases, and competitive requirements	Understands problem with minor gaps in constraints	Partial understanding; misses key constraints	Misinterprets problem or constraints

Algorithm	25%	Selects optimal algorithm with best time and space complexity for competitive constraints	Uses efficient algorithm with minor scope for improvement	Uses basic/inefficient approach	No clear algorithmic approach
Code Implementation	20%	Writes correct, efficient, and clean code that passes all constraints	Code works with minor errors or inefficiencies	Partially correct code; fails for some cases	Code does not execute or gives incorrect results
Competitive Performance	20%	Solves problem within time limits and handles large inputs effectively	Solves within acceptable time with minor delays	Struggles with time limits or larger inputs	Unable to solve within time constraints
Test Case Handling	15%	Handles all test cases including edge and hidden cases	Handles most test cases	Misses important edge cases	Fails on multiple test cases

Q. No. / Criteria	**Problem Understanding**	**Algorithm**	**Code Implementation**	**Competitive Performance**	**Test Case Handling**	**Sub. Total**	**Total %**
1							
2							
3							

Faculty In-charge	**Course Coordinator**	**Program Director**
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____